LECTURE 3

Virtualization Security

This lecture introduces the security of virtualized environments. It begins with the traditional server architecture that motivated virtualization, and then explains virtual machines, hypervisors, and the main benefits that virtualization provides. It next considers how cloud virtualization is secured, giving particular attention to hardware-based confidential computing through AMD SEV and Intel SGX and to the limitations of memory encryption. The lecture closes with four major virtualization threats: virtual machine escape, side-channel attacks, server-side request forgery, and insecure configuration.

IN THIS LECTURE

3.1 Traditional Server Architecture Without Virtualization
3.2 Virtualization and the Hypervisor
3.3 Benefits of Virtualization
3.4 Cloud Virtualization Security
3.5 Confidential Computing in Cloud Virtualization
3.6 Major Virtualization Threats

3.1 Traditional Server Architecture Without Virtualization

In a traditional server environment without virtualization, an organization can have multiple physical servers, each dedicated to providing a different service to clients. **Each physical server has its own hardware resources (CPU, RAM, and storage), an operating system installed directly on the hardware, and applications responsible for providing a particular service.** For example, one physical server may host a website, another may manage email services, and a third may store and manage database information. Clients communicate with these servers over a network to request and receive the required services (Figure 3.1).

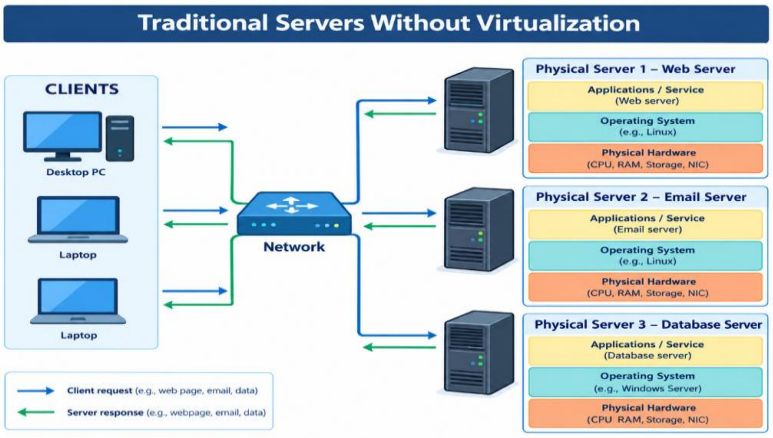


Figure 3.1 *A traditional server, with one operating system running on one set of hardware*

Although this approach provides dedicated resources for each service, it can lead to **hardware underutilization, increased costs, higher power consumption, and additional maintenance**, because each service requires separate physical hardware. These limitations

motivated the development of virtualization, which allows multiple virtual servers to run on a single physical machine.

3.2 Virtualization and the Hypervisor

3.2.1 Virtual Machines

Virtualization is a technology that allows multiple operating systems to run on a single physical machine by creating separate virtual computers called **virtual machines (VMs)**. A virtual machine behaves like a physical computer: it has its own virtual CPU, memory (RAM), storage, operating system, and applications. However, instead of having dedicated physical hardware, each VM uses a portion of the resources allocated to it from the same physical machine.

3.2.2 The Hypervisor

The **hypervisor** is a special software layer responsible for creating and running virtual machines, allocating physical CPU and RAM resources to each VM, and managing access to the physical hardware.

> **PRACTICAL EXAMPLE**
> Imagine a company has one powerful physical server. Instead of using three separate physical servers, it creates three virtual machines. VM 1 runs Linux and provides web services, VM 2 runs Linux and provides email services, and VM 3 runs Windows Server and manages databases. The hypervisor manages the physical resources and makes them available to these three VMs.

There are two types of hypervisors, which differ in where they run relative to the physical hardware.

3.2.3 Type 1 Hypervisors

A **Type 1 hypervisor** runs directly on the physical hardware without requiring a general-purpose operating system underneath it. It controls the physical hardware resources and allocates them to the virtual machines running on it. When a physical server starts, the hardware becomes available, and the Type 1 hypervisor runs directly on it to create and manage multiple virtual machines, each of which runs its own operating system and applications (Figure 3.2).

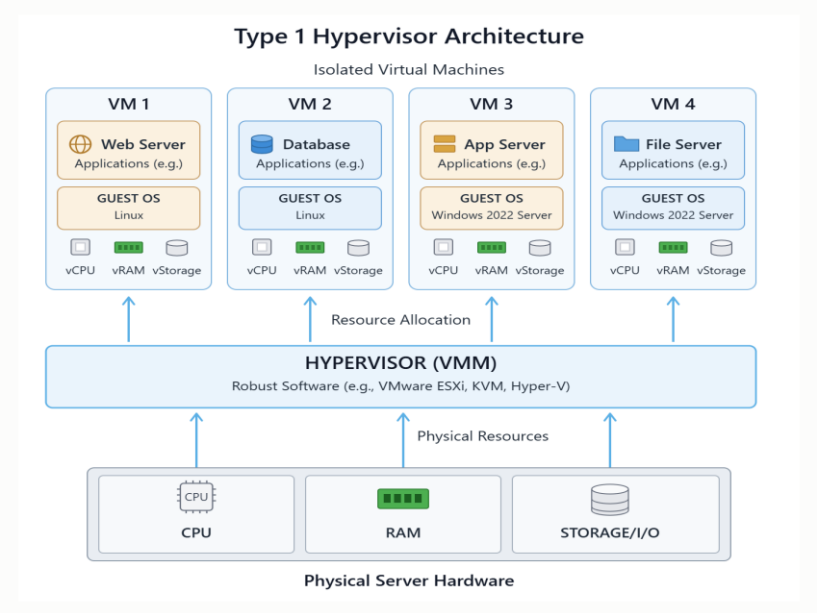


Figure 3.2 *Four VMs running on a server with a Type 1 hypervisor*

3.2.4 Type 2 Hypervisors

Unlike Type 1, a **Type 2 hypervisor** runs as an application inside an existing operating system. The physical computer already has an operating system installed, and the hypervisor is then installed like an ordinary application. The operating system and the hypervisor work together to provide the resources needed by the virtual machines (Figure 3.3).

This arrangement has four main layers. The **physical hardware** is the actual CPU, RAM, storage, and network devices. The **host operating system** is the operating system installed directly on the physical computer. The **Type 2 hypervisor** is a program installed on the host OS that creates and manages the VMs. Finally, the **guest operating systems** are the operating systems installed inside the virtual machines.

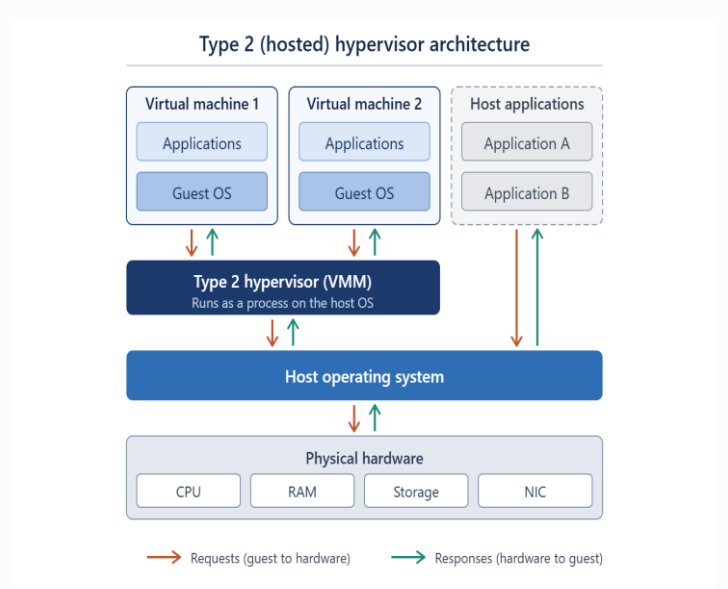


Figure 3.3 *A Type 2 hypervisor running as a program on a host OS, alongside other apps*

3.3 Benefits of Virtualization

Virtualization provides several important benefits that make managing computers and servers easier and more efficient. There are four fundamental benefits: partitioning, isolation, encapsulation, and hardware independence.

3.3.1 Partitioning

Partitioning means dividing the resources of one physical computer among multiple virtual machines. The hypervisor allocates resources such as CPU, RAM, and storage to each VM, so that each VM operates as an independent computer even though all of them share the same physical machine.

3.3.2 Isolation

Isolation means that each virtual machine operates separately from the other virtual machines. A problem inside one VM therefore does not normally affect the operation of the others. This is called **fault isolation**. Isolation also helps separate workloads and limit the impact of certain security incidents. However, it does not provide absolute protection against hypervisor vulnerabilities or attacks on shared hardware.

3.3.3 Encapsulation

Encapsulation means that a virtual machine and its configuration can be stored as files. This makes it easier to copy, back up, move, and restore virtual machines.

3.3.4 Hardware Independence

Hardware independence means that a virtual machine is not permanently tied to one specific physical computer. Because the VM uses virtual hardware provided by the hypervisor, it can often be moved to another compatible physical server without reinstalling its operating system and applications.

3.4 Cloud Virtualization Security

Cloud virtualization is generally secure because cloud providers invest heavily in protecting their infrastructure. However, security is not guaranteed. Attackers may exploit weaknesses in virtualization software, CPU hardware, cloud applications, or security configurations.

Three mechanisms contribute to the security of cloud virtualization. **Hardware isolation** means that each virtual machine operates in an isolated memory space, which prevents it from directly accessing another VM's memory. **Hypervisor security** relies on the hypervisor to control the virtual machines and separate them from one another, while security patches fix weaknesses before attackers exploit them. **Confidential computing** uses technologies such as AMD SEV and Intel SGX to protect certain data during processing, reducing the risk of unauthorized access.

3.5 Confidential Computing in Cloud Virtualization Security

This section focuses on two important hardware-based technologies: AMD Secure Encrypted Virtualization (AMD SEV) and Intel Software Guard Extensions (Intel SGX). Both help protect sensitive data while it is being processed, but they protect different things and use different key-management mechanisms.

The central concepts are memory encryption, hardware-protected keys, and the security boundary. AMD SEV protects the memory of an entire virtual machine, whereas Intel SGX protects selected application code and data inside an enclave.

3.5.1 Introduction to Confidential Computing

Cloud systems must protect data in three states. **Data at rest** is data stored on disks, databases, or other storage devices. **Data in transit** is data transferred between computers or services over a network. **Data in use** is data being processed by a CPU or another processing unit, usually while held in memory.

Traditional encryption methods, such as disk encryption and TLS, protect data at rest and in transit. However, data often needs to be decrypted before a processor can use it. This creates a potential exposure if the host operating system, hypervisor, or another privileged component is compromised.

Confidential computing addresses this problem by using hardware-based security mechanisms to protect sensitive data during processing. It establishes a protected execution environment in which selected data and code are isolated from software that is not trusted. In cloud virtualization, this is particularly important because multiple customers may share the same physical server. A cloud provider's hypervisor manages the VMs, but customers may not want to rely entirely on the hypervisor to keep their data confidential.

3.5.2 Hardware-Based Memory Encryption

Memory encryption protects data stored in physical memory (RAM) from unauthorized access by encrypting it before it is written to memory and decrypting it when it is read.

3.5.2.1 How Memory Encryption Works

A simplified memory-encryption process consists of the following operations.

1. **Data is processed by a VM.** Applications and the guest operating system use data in the VM's memory.
2. **Encryption before storage.** When data is written from the processor to physical memory, a hardware memory-encryption engine encrypts it.
3. **Encrypted storage.** The data is stored in RAM in encrypted form.
4. **Decryption during access.** When the processor reads the data, the memory-encryption engine decrypts it before making it available for processing.
5. **Transparent operation.** Encryption and decryption occur automatically in hardware, so applications generally do not need to perform these operations themselves.

The objective is to prevent unauthorized software or physical memory inspection from reading sensitive information directly from RAM.

3.5.2.2 The AES Encryption Algorithm

The **Advanced Encryption Standard (AES)** is a widely used symmetric encryption algorithm, which means that the same secret key is used for encryption and decryption. AES supports three key sizes, with the number of rounds shown in Table 3.2.

Table 3.2 *AES key sizes and number of rounds*

AES key size	Number of rounds
AES-128	10

AES key size	Number of rounds
AES-192	12
AES-256	14

AES processes data in fixed-size blocks of 128 bits. Its main encryption operations are **SubBytes**, which replaces each byte using a substitution table; **ShiftRows**, which shifts rows of the data block to rearrange the bytes; **MixColumns**, which combines bytes within each column to provide diffusion; and **AddRoundKey**, which combines the data block with a round key using an XOR operation. The final AES round does not include MixColumns.

3.5.2.3 What Is an Encryption Key?

An **encryption key** is a secret value used by an encryption algorithm to transform readable data into encrypted data and to recover the original data. The memory-encryption engine, for example, uses a secret key to encrypt and decrypt memory contents. Without the correct key, an unauthorized party should not be able to recover the original plaintext from the encrypted memory contents, which is why the key must be protected: an attacker who obtains it may be able to decrypt the protected data. It is also important to distinguish between the encryption algorithm and the encryption key. AES is the algorithm; the secret key is the value used by AES.

3.5.3 Hardware-Based Key Generation and Management

Memory encryption depends on secret keys, so protecting the encryption key is just as important as encrypting the memory itself. In confidential computing, keys are managed by hardware security components rather than being freely accessible to the operating system or hypervisor.

3.5.3.1 How Are Encryption Keys Generated and Protected?

A hardware security mechanism typically performs the following functions.

1. **Key generation.** Creates a cryptographically secure secret key.
2. **Key assignment.** Associates the key with a particular VM or protected execution environment.
3. **Key protection.** Keeps the key inside a protected hardware boundary.
4. **Key use.** Allows the relevant hardware encryption mechanism to use the key without exposing it to untrusted software.
5. **Key isolation.** Prevents one VM or protected environment from using another environment's key.

Three related terms should be distinguished, as Table 3.3 shows. The exact mechanism differs between AMD SEV and Intel SGX.

Table 3.3 *Key generation, key derivation, and key management*

Term	Meaning
Key generation	Creating a new secret key.
Key derivation	Producing a specific key from a protected root key and other inputs, such as an enclave identity.
Key management	Controlling the key's assignment, use, protection, lifetime, and access.

3.5.4 AMD SEV: Protecting the Memory of an Entire Virtual Machine

AMD Secure Encrypted Virtualization (AMD SEV) is a hardware-based security technology designed to protect the memory of a virtual machine from unauthorized access by the hypervisor

or other software running outside that VM. Unlike application-level protection, AMD SEV aims to protect the memory of the entire guest VM.

3.5.4.1 Main Components of AMD SEV

The AMD SEV architecture involves four important components. The **guest virtual machine** contains its own applications, guest operating system, and memory, and under AMD SEV its memory is encrypted when stored in physical memory. The **hypervisor** creates and manages VMs, allocating resources and controlling VM execution, but it is not supposed to have access to the plaintext contents of an SEV-protected VM's memory.

The **AMD Secure Processor (AMD-SP)** is a dedicated security component that performs security-related functions for SEV. It initializes and manages SEV security functions, generates or manages VM-specific memory-encryption keys, associates protected VMs with their encryption contexts, and keeps sensitive key material within a protected hardware environment. The hypervisor can request certain operations through the supported secure interface, but it cannot directly read the protected VM encryption key. Finally, the **memory encryption engine** is implemented in the processor's memory-access path and automatically encrypts and decrypts VM memory as data moves between the processor and physical memory.

3.5.4.2 How AMD SEV Memory Encryption Works

The process can be explained in the following steps.

1. **Initialize the security mechanism.** The AMD Secure Processor initializes the SEV functionality and its firmware.
2. **Create a protected VM.** The hypervisor creates a VM and requests that it be protected using SEV.
3. **Associate an encryption key.** A VM-specific memory-encryption key is generated or assigned through the hardware security mechanism.
4. **Protect the key.** The key remains protected and is not exposed to the hypervisor as plaintext.
5. **Encrypt VM memory.** When the VM writes data to physical RAM, the memory-encryption engine encrypts the data using the VM's associated key.
6. **Decrypt VM memory.** When the VM reads its data, the processor decrypts it automatically before providing it to the VM.
7. **Restrict unauthorized access.** If the hypervisor attempts to inspect the encrypted VM memory directly, it cannot obtain the original plaintext data.

3.5.4.3 The VM Encryption Key (VEK)

The **Virtual Machine Encryption Key (VEK)** is the secret key associated with an SEV-protected VM. It is used by the hardware memory-encryption mechanism to protect that VM's memory. The processor uses the appropriate key when accessing each VM's memory, which helps prevent the hypervisor or another VM from reading the protected memory contents.

> **IMPORTANT DISTINCTION**
> ASID is not an encryption key. An Address Space Identifier (ASID) is an identifier used by the processor to distinguish address-space or encryption contexts. In SEV, it helps associate memory accesses with the appropriate VM encryption context. It does not encrypt data and is not a secret encryption key.

3.5.4.4 AMD SEV Security Extensions

AMD SEV has evolved through several extensions, summarized in Table 3.4.

Table 3.4 *AMD SEV extensions and their main protection*

Technology	Main protection
SEV	Encrypts VM memory to protect its confidentiality from the hypervisor.
SEV-ES	Adds protection for sensitive CPU register state when a VM exits to the hypervisor.
SEV-SNP	Adds memory integrity-related protections, including protection against certain malicious memory remapping and replay attacks.

SEV-SNP is important because basic memory encryption primarily protects confidentiality. Encryption alone does not necessarily prevent an attacker from modifying encrypted memory or replaying previously observed memory contents. SEV-SNP adds protections intended to help detect or prevent certain forms of malicious memory manipulation, and it therefore strengthens the security of confidential VMs beyond basic memory confidentiality.

3.5.5 Intel SGX: Protecting Selected Application Code and Data

Intel Software Guard Extensions (Intel SGX) is a hardware-based technology that protects selected application code and data inside a protected execution environment called an **enclave**. Unlike AMD SEV, which protects the memory of an entire VM, Intel SGX protects only the application components placed inside an enclave.

3.5.5.1 Main Components of Intel SGX

Intel SGX involves four main components. The **application outside the enclave** is the ordinary application code that runs outside the protected environment; it may communicate with the enclave but is not automatically trusted. The **enclave** is a protected memory region containing selected application code and sensitive data. An application may, for example, place its cryptographic operations and secret keys inside an enclave while keeping other application functions outside it.

The **Enclave Page Cache (EPC)** is a protected memory region used to hold enclave pages, and hardware security mechanisms protect the enclave's contents against unauthorized access by the operating system or hypervisor. The **CPU and SGX key hierarchy** consists of hardware-protected key material in the processor that is used to derive keys for SGX security operations. These keys are not directly exposed to the operating system or hypervisor.

3.5.5.2 How Intel SGX Protects Data

The process works as follows.

1. **Create an enclave.** The application creates a protected memory region called an enclave.
2. **Load protected code and data.** Selected application code and sensitive information are placed inside the enclave.

3. **Establish enclave identity.** The processor establishes information about the enclave's identity and security attributes.
4. **Execute protected code.** The application enters the enclave using an ECALL (Enclave Call).
5. **Process sensitive information.** The enclave performs sensitive operations, such as cryptographic calculations, using protected data.
6. **Return results.** The enclave exits through an EEXIT operation and returns permitted results to the application.
7. **Protect enclave contents.** The hardware prevents ordinary software, including the operating system and hypervisor, from directly reading the enclave's protected memory.

3.5.5.3 How Are Keys Generated and Used in Intel SGX?

Intel SGX uses a hardware-protected key hierarchy. Rather than allowing the operating system or application to create and freely access all enclave security keys, the processor derives keys from hardware-protected secret material. One important instruction is **EGETKEY**, which an enclave uses to request a key derived by the processor. The derived key depends on the requested key type and relevant enclave or platform security information. An enclave may, for example, request a sealing key to protect sensitive information that needs to be stored outside the enclave. The enclave then uses the derived key to encrypt or decrypt protected data, and the encrypted data can be stored outside the enclave.

> THE IMPORTANT POINT
> The CPU derives the requested key. The operating system and hypervisor do not receive the underlying hardware-protected root secret.

3.5.5.4 Key Sealing

Sealing is a mechanism that allows an enclave to encrypt sensitive data so that it can be stored outside the enclave and later recovered by an appropriately authorized enclave. The process involves five steps.

1. **Generate or hold a key.** An enclave generates or holds a sensitive cryptographic key.
2. **Request a sealing key.** The enclave requests a sealing key using EGETKEY.
3. **Encrypt.** The enclave encrypts the sensitive information using the derived sealing key.
4. **Store.** The encrypted information is stored outside the enclave, such as on a disk.
5. **Recover.** When the enclave needs the information again, it requests the appropriate sealing key and decrypts the stored data.

3.6 Major Virtualization Threats

Cloud virtualization allows multiple virtual machines to share the physical resources of a single server. A hypervisor manages these VMs and provides isolation between them. This arrangement delivers flexibility, scalability, and efficient use of resources, but it also introduces security risks. An attacker may exploit vulnerabilities in the hypervisor, abuse shared hardware resources, attack cloud applications, or take advantage of insecure configurations. If these weaknesses are not addressed, the attacker may access sensitive information, compromise other VMs, or gain control over parts of the cloud infrastructure.

This section examines three important threats, each of which targets a different part of the virtualization stack, as Figure 3.4 shows. **Virtual machine escape** attacks the isolation boundary enforced by the hypervisor. **Side-channel attacks** exploit the behavior of hardware that VMs

share. **Server-side request forgery (SSRF)** abuses a vulnerable application to reach internal cloud services.

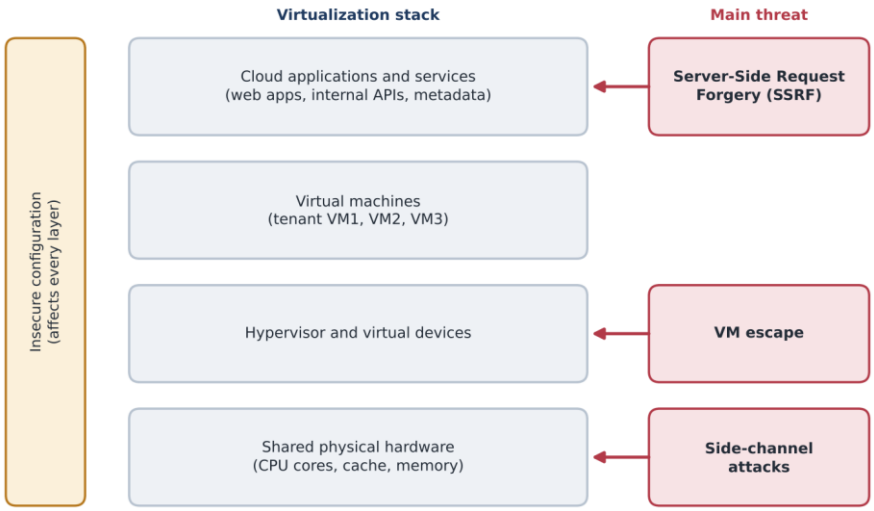


Figure 3.4 *The virtualization stack and the main threat that targets each layer*

3.6.1 Virtual Machine Escape

Definition

Virtual machine escape is an attack in which an attacker compromises a VM and then exploits a vulnerability in the hypervisor or virtualization software to gain access to the host system or, in some cases, to other VMs. Normally, a VM operates in an isolated environment, and its operating system and applications should not be able to access the host operating system or another VM directly. VM escape occurs when an attacker bypasses this isolation.

How the attack occurs

Consider a cloud server running a hypervisor that hosts three VMs belonging to different customers. The attack unfolds in five steps, numbered in Figure 3.5 and in the list below.

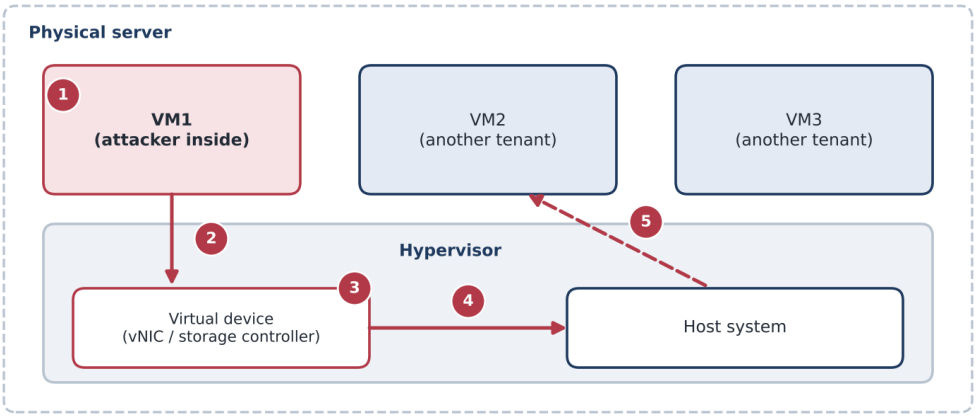


Figure 3.5 *VM escape: from a compromised guest VM to the host system*

1. **Compromise a VM.** An attacker exploits a vulnerability in an application or operating system running inside VM1.
2. **Exploit virtualization software.** After gaining control of VM1, the attacker sends specially crafted requests to a virtual device, such as a virtual network adapter or storage controller.

3. **Trigger a hypervisor vulnerability.** If the virtual device or hypervisor contains a security flaw, the malicious request may cause code to execute outside the VM's isolated environment.
4. **Escape the VM.** The attacker gains unauthorized access to the host system or the hypervisor.
5. **Attack other resources.** Depending on the vulnerability and the attacker's privileges, the attacker may access other VMs, interfere with their operations, or obtain sensitive information.

3.6.2 Side-Channel Attacks

Definition

A **side-channel attack** obtains sensitive information by observing indirect information about how a computer system operates, rather than by accessing the protected data directly. In cloud virtualization, multiple VMs may share physical CPU cores, cache memory, and other hardware resources. Although the hypervisor separates their virtual environments, some hardware activity can still reveal information through observable differences in execution time or resource behavior.

How the attack occurs

Consider two VMs belonging to different customers that run on the same physical processor (Figure 3.6).

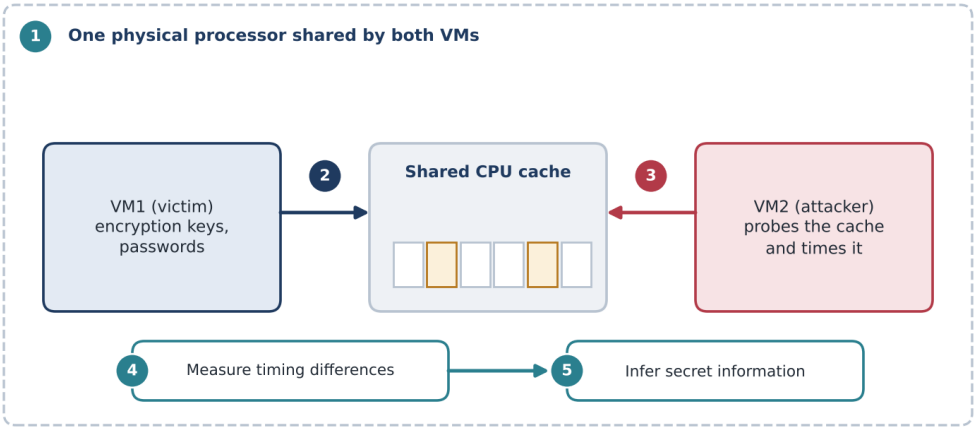


Figure 3.6 *A cache side channel between two VMs on the same processor*

1. **Share physical hardware.** The cloud provider runs VM1 and VM2 on the same physical CPU.
2. **Execute sensitive operations.** VM1 processes sensitive information, such as encryption keys or passwords.
3. **Observe shared hardware behavior.** The attacker controls a process inside VM2 and observes changes in shared CPU resources, such as cache activity.
4. **Measure execution differences.** The attacker measures how long certain operations take or observes cache access patterns.
5. **Infer sensitive information.** By analyzing repeated measurements, the attacker may infer information about VM1's operations or potentially recover secret data.

Side-channel techniques

Cache attacks observe CPU cache access patterns and timing differences. **Timing attacks** measure execution time to infer information about sensitive operations. **Speculative-execution attacks** exploit CPU behavior to expose information through microarchitectural side effects.

3.6.3 Server-Side Request Forgery (SSRF)

Definition

Server-side request forgery (SSRF) is a web application vulnerability that allows an attacker to make a server send unauthorized requests to other systems or services. In cloud environments, an SSRF vulnerability may let an attacker reach internal services that are not directly accessible from the Internet, such as cloud management interfaces, internal APIs, or metadata services that provide information about the cloud instance. SSRF is primarily an application-layer vulnerability, but it becomes a serious cloud infrastructure threat when the vulnerable application has access to sensitive internal services.

How the attack occurs

Consider a cloud-hosted web application that lets users provide a URL so that the application can retrieve an image or download a file (Figure 3.7).

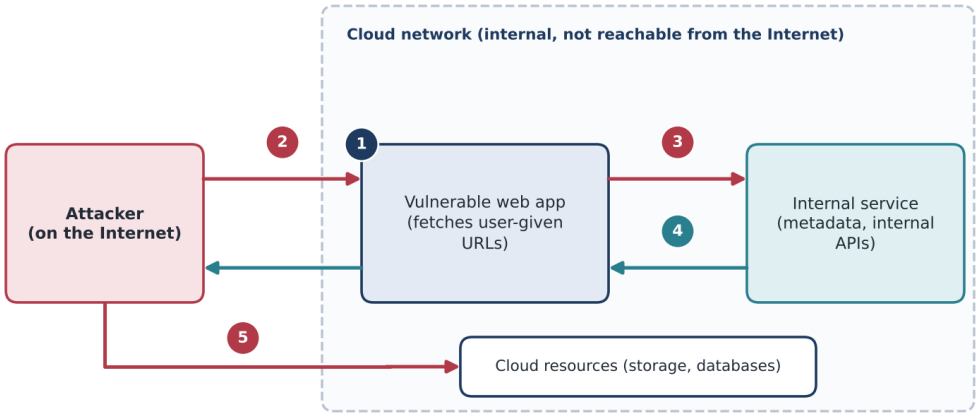


Figure 3.7 *SSRF: using a vulnerable server to reach internal cloud services*

1. **Identify a vulnerable application.** An attacker discovers a web application that accepts a URL from users and retrieves content from it.
2. **Provide a malicious URL.** Instead of a normal public address, the attacker supplies a URL that points to an internal cloud service.
3. **Cause the server to send a request.** The application processes the URL and makes a request from the server's own network environment.
4. **Access an internal service.** If the application can reach the requested service and the service does not adequately restrict access, the attacker may obtain information that should not be publicly accessible.
5. **Abuse the obtained information.** Depending on the service and the application's permissions, the attacker may retrieve sensitive metadata or credentials and use them to access additional cloud resources.